COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	OCTOBER 20 2026	**DATE SUBMITTED:**	OCTOBER 20 2026
NAME:	JARED ISAIAH DELA CRUZ DUSTIN CARL MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 14 **Hands-on Activity 14.1 Algorithm Complexity**

1. PROCEDURE OUTPUT

ALGORITHM 1 - BUBBLE SORT

```cpp
template <typename T>
void bubbleSort(T arr[], int maxSize) {
    for (int i = 0; i < maxSize; i++) {
        for (int j = i + 1; j < maxSize; j++) {
            if (arr[j] < arr[i]) {
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

ALGORITHM 2 - INSERTION SORT

```cpp
template <typename T>
void insertionSort(T arr[], int arrSize) {
    int k = 1, j;
    T temp;

    while (k < arrSize) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j]) {
            arr[j + 1] = arr[j];
            j--; // back tracking part
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

	BEST CASE	WORST CASE	ANALYSIS
BUBBLE SORT	$O(n^2)$	$O(n^2)$	The outer loop runs n times and inner loop n-1, n-2, … times. Hence the comparison count is O(n^2). It is both the same for the worst and best case since it does not have any helper

			functions to check whether the array is already sorted or not.
INSERTION SORT	O(n)	O(n²)	The main idea of insertion sort is sorting the elements through insertion of each element into its new and correct position in the subarray, showcasing O(n^2) in the worst case and O(n) in the best case when arranged.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	BUBBLE SORT	**INSERTION SORT**
BEST CASE	**N = 100** Bubble Sort (Best Case) | Comps: 4950 | Time: 7 us **N = 1000** Bubble Sort (Best Case) | Comps: 499500 | Time: 774 us **N = 10000** Bubble Sort (Best Case) | Comps: 49995000 | Time: 84242 us	**N = 100** Insertion Sort (Best Case) | Comps: 99 | Time: 0 us **N = 1000** Insertion Sort (Best Case) | Comps: 999 | Time: 1 us **N = 10000** Insertion Sort (Worst Case) | Comps: 499500 | Time: 842 us
WORST CASE	**N = 100** Bubble Sort (Worst Case) | Comps: 4950 | Time: 9 us **N = 1000** Bubble Sort (Worst Case) | Comps: 499500 | Time: 901 us **N = 10000** Bubble Sort (Worst Case) | Comps: 49995000 | Time: 107482 us	**N = 100** Insertion Sort (Worst Case) | Comps: 4950 | Time: 7 us **N = 1000** Insertion Sort (Worst Case) | Comps: 499500 | Time: 842 us **N = 10000** Insertion Sort (Worst Case) | Comps: 49995000 | Time: 84429 us

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis

PSEUDOCODE	
BINARY SEARCH	**LINEAR SEARCH**

```
Algorithm BinarySearch(A, N, target):
    Input: Sorted array A of size N, search value target
    Output: Index of target if found, otherwise -1

    left = 0
    right = N - 1

    While left <= right do:
        mid = left + floor((right - left) / 2)

        If A[mid] == target then:
            Return mid
        Else If A[mid] < target then:
            left = mid + 1
        Else:
            right = mid - 1
        End If
    End While

    Return -1
```

```
Algorithm LinearSearch(A, N, target):
    Input: Array A of size N, search value target
    Output: Index of target if found, otherwise -1

    For i = 0 to N - 1 do:
        If A[i] == target then:
            Return i
        End If
    End For

    Return -1
```

INPUT SIZE (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.022300 ms	1.544400 ms
10,000	0.038600 ms	15.332300 ms
100,000	0.044900 ms	157.263800 ms

THEORETICAL ANALYSIS: Binary search has a worst case of O(log n)and a best case of O(1) because it repeatedly divides the search in half, and it has a space complexity of O(1)f. While linear search has the worst case of O(n)and a best case of O(1) since this algorithm must check every element one by one, and its space complexity is also O(1). Furthermore in the pseudocode, binary search needs the array to be sorted out already in order for it to work. And it repeatedly cuts the search in half to calculate the midpoint. While the pseudocode for the linear search is that it can work even if the array is not sorted, and it will go through or check all the elements inside the array when searching for a target. If the target is already found then it will stop.

EMPIRICAL ANALYSIS:

INPUT SIZE (N)	Binary Search	Linear Search
1000	0.022300 ms	1.544400 ms
10000	0.038600 ms	15.332300 ms
100000	0.044900 ms	157.263800 ms

CONCLUSION

DELA CRUZ: Personally, this activity felt like another showcase on how different sorting algorithms work within different case scenarios. All we did for this activity is tweak the program so that it would only focus on getting the speed in which the results would be gained in the specified case scenario, which is the worst case for the supplementary activity. However, through this activity, we were able to determine which sorting algorithm is superior for both cases, which helped us have a deeper understanding on how each algorithm works under special conditions and to visually see the gap between their return speed. It also gave us a better insight on how their time complexities are perceived.

MAPAS: In this hands-on activity, I got a recall about asymptotic notations. Moreover, I also learn about algorithm complexity, and what can affect run time of an algorithm. In addition, I also created a code for testing and comparison of two searching and sorting algorithms. We also identified the best and worst case of those algorithms and then provided an analysis. By measuring those cases we can identify how efficiently that algorithm runs and provide an analysis which algorithm to use. Lastly, this activity was like a review about the different best and worst cases of the different searching and sorting algorithms, and it provided me with a good insight about algorithm complexity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction. Code analysis and Verification**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We only used AI as stated in its purpose of use. We strictly used AI only for assistance and understanding of the activity. All the content included in this document remains original and of our own authorship.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.